GIZMO+

Technical Traceability & Verification Specification

Companion to the GIZMO+ Product & Technical Blueprint

Purpose
This document turns the blueprint into a verifiable build contract. It distinguishes legacy evidence from redesign decisions, maps requirements through database/services/Mastra/CopilotKit/tests, and defines deterministic AI verification with AIMock. It also freezes Better Auth as the authentication and workspace-membership baseline.

Architecture baseline: Better Auth + Turso/libSQL + Mastra + CopilotKit/AG-UI + AIMock

Prepared September 2026 | *Demo data is entirely fictional.*

1. How to Use This Specification

The Product & Technical Blueprint explains what GIZMO+ is and why the system is designed the way it is. This companion answers a different question: can every important behavior be traced to evidence, a design decision, an executable contract, and a test?

- Coding agents should treat requirement IDs in this document as stable references in implementation plans, commits, tests, and pull requests.
- No agent may silently reinterpret a VERIFIED legacy rule. If a rule must change, create an architecture decision record (ADR) and update its traceability row.
- Items marked PROPOSED are not equivalent to user-verified requirements. They are design proposals that may be implemented only when they do not contradict VERIFIED requirements.
- An item is not considered implemented merely because a UI exists. Data authorization, persistence, agent tool behavior, UI rendering, and automated verification must all be satisfied.

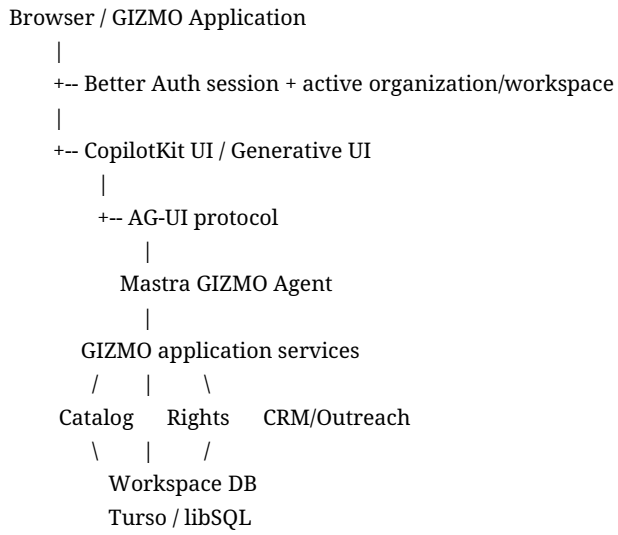
2. Verification Vocabulary

Level	Meaning	Evidence required
L0 - Proposed	Architecture or product idea only	Reasoned design note; no source or executable proof required yet.
L1 - Source Verified	Confirmed against Airtable, official product docs, or authoritative music-domain source	Source reference and exact legacy field/rule or official documentation.
L2 - Contract Specified	Implementation contract is explicit	Schema fields, service contract, tool I/O, component mapping, auth rule, and test ID.
L3 - Implementation Verified	Working implementation demonstrates contract	Automated unit/integration test plus executable code.
L4 - E2E Verified	Real user workflow passes through entire stack	Browser/E2E test or equivalent full-stack verification.

3. Evidence Classification

Class	Definition	Example
LEGACY-VERIFIED	Directly present in GIZMO/GIZMO+ Airtable	SONGS has separate rollups for writer, publishing, and master ownership totals.
USER-DECISION	Explicit architecture/product decision	Use Turso/libSQL instead of Supabase; Better Auth for auth; Mastra for agent runtime.
DOC-VERIFIED	Confirmed in current official product documentation	CopilotKit can render Mastra tool calls with custom UI components.
DOMAIN-CORRECTION	Modern model intentionally corrects a legacy simplification	Composition and sound recording become distinct entities.
PROPOSED-ENHANCEMENT	Useful capability not proven as legacy requirement	Pitch status history, follow-up due dates, richer campaigns.
UNRESOLVED	Intentionally deferred decision	Object-storage and email-delivery vendor selection.

4. Frozen Architecture Baseline



Control plane (separate): Better Auth + workspace registry
Agent runtime (separate): Mastra storage on libSQL/Turso
Verification layer: AIMock + AGUIMock + Vitest + E2E

Critical separation
Three logical data responsibilities must not collapse into one undifferentiated database: (1) Better Auth/control-plane identity and workspace registry, (2) canonical GIZMO workspace music/business data, and (3) Mastra agent runtime state such as threads, memory, workflow snapshots, and evaluation data.

5. Better Auth Decision

Better Auth is now the authentication and authorization baseline. Current Better Auth documentation describes built-in session/account management plus an Organization plugin for organizations, members, teams, invitations, active organization, roles, and permissions. It supports SQLite and documents libSQL/sqld as a Kysely-compatible relational dialect. The exact libSQL adapter wiring remains an L2 implementation spike until proven in the GIZMO repository.

ID	Requirement / decision	Evidence	Target level
AUTH-001	All authenticated application requests resolve a server-validated Better Auth session.	USER-DECISION + Better Auth docs	L4
AUTH-002	GIZMO workspace membership is represented by Better Auth Organization membership or an explicitly mapped equivalent.	USER-DECISION + Better Auth Organization docs	L4
AUTH-003	The current/active organization determines the GIZMO workspace context.	DOC-VERIFIED concept; implementation choice	L4
AUTH-004	Authorization is enforced server-side at GIZMO service boundaries; hiding UI controls	Security requirement	L4

	is not authorization.		
AUTH-005	Organization/workspace invitations, roles, and membership changes are auditable.	PROPOSED but recommended	L3
AUTH-006	Better Auth schema migrations are managed separately from per-workspace GIZMO migrations.	Architecture decision	L3

Recommended control-plane model: one central auth/control database stores Better Auth tables plus a workspace_registry mapping Better Auth organization IDs to Turso workspace database identifiers. Per-workspace music data does not live in Better Auth tables.

6. Turso / libSQL Tenancy Decision

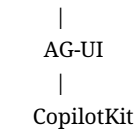
The preferred deployment is database-per-workspace, not a shared application database with workspace_id on every business table. Turso currently promotes database-per-tenant patterns and supports large numbers of databases. Its older Multi-DB Schemas feature is deprecated for new users, so GIZMO should own schema migration orchestration rather than depend on that feature.

ID	Contract	Verification
TEN-001	Each GIZMO workspace maps to exactly one canonical workspace database.	Integration test provisions two workspaces and proves cross-workspace data cannot be queried through either workspace connection.
TEN-002	A central workspace registry resolves organization_id -> database identifier/URL metadata.	Integration test.
TEN-003	Every workspace database carries an explicit schema_version.	Migration test.
TEN-004	New workspace provisioning applies the current canonical schema deterministically.	Provisioning integration test.
TEN-005	Migrations can be applied across all workspace databases with resumable tracking and failure reporting.	Migration runner test.
TEN-006	Mastra runtime storage is not the canonical GIZMO workspace database.	Architecture/test configuration assertion.

7. AIMock as a First-Class Verification Layer

AIMock is now part of the required test architecture. Its current Mastra guide documents deterministic LLM responses, tool-call fixture sequences, Vitest integration, AGUIMock for CopilotKit/AG-UI event streams, and CI operation. This gives GIZMO a way to test agent behavior without token cost, provider variability, or a running Mastra backend for every frontend test.

Production / exploratory evals
  Real model provider
    |
  Mastra



Deterministic CI path

AIMock LLM fixtures ---> Mastra tool planning tests
AGUIMock events ------> CopilotKit rendering tests
Seeded Turso DB ------> Service/integration tests
Browser E2E ----------> Full workflow verification

ID	AIMock requirement	Acceptance proof
TST-AI-001	Mastra agent tests can redirect compatible model traffic to AIMock.	Vitest suite runs without external model key.
TST-AI-002	Fixtures cover single-tool and multi-tool sequences.	Fixture tests for search -> retrieve -> render and draft -> confirm -> mutate.
TST-AI-003	AGUIMock can drive CopilotKit components without Mastra running.	Component integration test renders SongCard, RightsPanel, PitchCard, approval UI.
TST-AI-004	AIMock fixtures are versioned in the repository.	CI checks fixture directory and schema.
TST-AI-005	At least one separate real-model evaluation suite remains; mocks do not replace model-quality evaluation.	Scheduled/manual eval job.
TST-AI-006	CI blocks merges when deterministic agent/UI regression tests fail.	Required CI check.

8. Requirement Traceability Matrix - Core Product

ID	Requirement	Source	DB	Service	Mastra	UI	Test
CAT-001	Create and edit a composition/song	Airtable SONGS.Song Name + metadata	compositions	CompositionService	create/ updateComposition	CompositionForm, CompositionCard	CAT-E2E-001
CAT-002	Attach one or more recordings to a composition	DOMAIN-CORRECTION from mixed SONGS model	recordings	RecordingService	create/ getRecording	RecordingCard	CAT-E2E-002
CAT-003	Represent primary and featured artists	SONGS.Artist / Featuring	recording_artists	RecordingService	updateRecording Artists	ArtistChips	CAT-INT-003
CAT-004	Store identifiers and musical metadata	SONGS ISRC/ISWC/BPM/ Key/Genre/etc.	compositions + recordings	CatalogService	get/update metadata	MetadataPanel	CAT-E2E-004
CAT-005	Store listening/previe w details	Listen Link / Preview StartAt	recording_assets or recordings	AssetService	getListenAsset	AudioPlayer	CAT-E2E-005
RGT-001	Unlimited songwriter shares per composition	Songwriting junction	composition_wri ter_splits	RightsService	add/ updateWriterSpl it	RightsPanel	RGT-E2E-001
RGT-002	Unlimited publisher shares	Publishing junction	publishing_splits	RightsService	add/ updatePublisher	RightsPanel	RGT-E2E-002

	per composition				Split		
RGT-003	Unlimited master owners per recording	MasterOwner junction	master_owner_splits	RightsService	add/ updateMasterSplit	RightsPanel	RGT-E2E-003
RGT-004	Validate ownership totals	Airtable Writers/Publishing/Master Total % rollups	derived query/view	RightsService	validateOwnership	OwnershipValidator	RGT-E2E-004
CRM-001	Store industry contacts and companies	Clients table	contacts + organizations	CRMService	create/getContact	ClientProfile	CRM-E2E-001
CRM-002	Track a song/contact pitching relationship	Pitching Status table	pitches	PitchService	create/getPitch	PitchCard	CRM-E2E-002
CRM-003	Filter pitches by status	Four legacy status choices	pitches.status	PitchService	searchPitches	PitchTable/ Kanban	CRM-E2E-003
OUT-001	Draft outreach from pitch + catalog context	Email table	messages/drafts	OutreachService	draftPitchEmail	EmailDraftEditor	OUT-E2E-001
RPT-001	Generate configurable metadata report	SONGS.Report formula + toggles	templates + derived data	ReportService	generateMetadataReport	ReportPreview	RPT-E2E-001
RPT-002	Generate no-lyrics and lyrics-only variants	ReportSansLyrics / ReportJustLyrics	report templates	ReportService	generateReportVariant	ReportPreview	RPT-E2E-002

9. Airtable-to-Modern-System Traceability Rules

- Airtable formulas used only for display formatting become UI/service formatting, not persisted canonical fields.
- Airtable lookup and rollup fields become joins/queries/views unless persistence is required for historical snapshots.
- Airtable record links that represent many-to-many relationships become explicit relational junction tables.
- The legacy SONGS row mixes composition and sound-recording concepts. Migration must split those concerns without losing original fields.
- Legacy report checkboxes are interpreted as report-section configuration, not permanent song attributes.
- Legacy client status lookup columns (Hasn't Heard, Awaiting Reply, Passed On, Currently Pitching) become filtered queries over pitch status, not columns on contacts.
- Legacy Email lookup fields become joins from message -> pitch -> contact/recording rather than duplicated data.

10. Canonical Entity Contract

Entity	Purpose	Key relationships
person	Human creative, contact, songwriter, artist, producer, engineer, etc.	May hold multiple roles; linked to organizations and rights relationships.
organization	Publisher, label, sync agency, company, master owner, etc.	Contacts, rights, releases, pitches.
composition	Underlying musical work/song.	Writers, publishers, alternate titles, ISWC/copyright metadata, recordings.
recording	Specific sound recording/master of a composition.	Artists, master owners, ISRC, duration/BPM/key/audio assets, release tracks.

release	Album/single/EP/product context.	Contains release tracks/recordings; UPC and release-level metadata.
pitch	Business relationship between one target/contact and one or more pitched recordings/compositions.	Status, activity history, messages, notes, follow-up.
message	Draft/sent outreach associated with a pitch and recipient.	Provider IDs, timestamps, body/subject, status.
report template	Defines sections and formatting for metadata package output.	Generated reports reference canonical data and optionally snapshot output.
asset	Metadata reference to audio, image, PDF, agreement, or exported report stored outside Turso.	Linked to recordings/releases/organizations/reports .

11. Mastra Agent Contract

The GIZMO agent may reason about which safe business operation to invoke, but it does not receive arbitrary SQL access. Services own authorization, transactions, validation, and database access. The agent is a caller of those contracts, not a privileged database administrator.

Tool	Risk	Purpose	HITL	Renderer
searchCompositions	read	Search composition/recording catalog	No	CompositionGrid
getComposition	read	Retrieve canonical composition with recordings/rights summary	No	CompositionDetail
createComposition	write	Create a composition	Yes when inferred data is ambiguous	CompositionCard
updateComposition	write	Update composition metadata	Confirm destructive/identifier changes	CompositionDetail
validateOwnership	read	Calculate writer/publisher/master totals and problems	No	OwnershipValidator
updateWriterSplit	write-sensitive	Change ownership percentage	Yes	RightsEditor
updatePublisherSplit	write-sensitive	Change publishing percentage	Yes	RightsEditor
updateMasterSplit	write-sensitive	Change master ownership percentage	Yes	RightsEditor
searchContacts	read	Find contacts/organizations	No	ClientList
createPitch	write	Create pitch relationship	Yes before external side effects	PitchCard
updatePitchStatus	write	Change pitch status + history	Optional for simple reversible update	PitchTimeline

draftPitchEmail	draft	Generate editable message draft	No external side effect	EmailDraftEditor
sendPitchEmail	external-side-effect	Send through configured provider adapter	Always	SendApprovalCard
generateMetadataReport	derived	Build report preview/export payload	No	ReportPreview

12. CopilotKit / AG-UI Rendering Contract

The conversational interface must return application UI, not merely prose, when a tool has a structured domain result. CopilotKit currently supports custom rendering for Mastra tool calls, shared state, and interrupts. These capabilities are used deliberately: deterministic domain results use app-owned components; human approvals use interrupts; shared state keeps the app and agent synchronized.

Result / action	Required UI behavior	Text-only response allowed?
searchCompositions	Render CompositionGrid or SongCards with actions	No, except zero-result explanation.
getComposition	Render CompositionDetail with metadata, rights summary, recordings, pitch summary	No.
validateOwnership	Render OwnershipValidator with totals, warnings, and fix actions	No.
searchPitches	Render PitchTable/Kanban or PitchCards	No.
draftPitchEmail	Render editable EmailDraftEditor	No.
sendPitchEmail	Render approval interrupt before side effect, then delivery result	No.
long-running workflow	Render progress/activity state from AG-UI shared state	Brief explanatory text may supplement UI.
simple explanation/question	Normal assistant text	Yes.

13. Authorization Matrix

Operation	Owner	Admin	Member	Viewer	Agent rule
Read catalog	Yes	Yes	Yes	Yes	Allowed if user can read active workspace.
Create/edit catalog	Yes	Yes	Yes	No	Service re-checks role.
Delete catalog records	Yes	Yes	No*	No	Require explicit confirmation; *policy can be relaxed later.
Edit rights/splits	Yes	Yes	Yes*	No	Always service-authorized; sensitive mutations

					require confirmation.
Create/update pitches	Yes	Yes	Yes	No	Service-authorized.
Send external email	Yes	Yes	Yes*	No	Always approval + service authorization.
Invite members	Yes	Yes	No	No	Better Auth Organization permission.
Change workspace roles	Yes	Yes*	No	No	Cannot remove final owner; Better Auth permission.
Delete workspace	Yes	No	No	No	Destructive confirmation + control-plane audit.

14. Demo ICPs and Fictional Seed Personas

The demo should not feel like an empty CRUD application. It should contain multiple realistic ideal-customer-profile scenarios so agent queries, UI components, rights validation, pitching, and outreach can all be demonstrated. All names, companies, domains, songs, phone numbers, and identifiers below are fictional test data.

ID	Name	ICP	Company	Email	Phone	Demo story
ICP-01	Jordan Taylor	Independent singer-songwriter	Silverline Songs	jordan@silverline.example	+1 213-555-0101	Owns catalog, manages metadata/rights personally, pitches songs to sync contacts.
ICP-02	Alex Ramirez	Producer / co-writer	Nightshift Audio	alex@nightshift.example	+1 323-555-0112	Works across multiple artists and needs clean writer splits, producer credits, and versions.
ICP-03	Rebecca Carter	Sync licensing agent	Carter Sync Agency	rebecca@cartersync.example	+1 310-555-0127	Represents catalogs, tracks one-stop readiness, metadata packages, pitches, and supervisor follow-up.
ICP-04	Malik Johnson	Independent label operator	NextWave Records	malik@nextwave.example	+1 424-555-0134	Manages artists, master ownership, releases, licensing contacts, and outbound pitching.
ICP-05	Sophia Lee	Artist manager / pitching operator	Northstar Management	sophia@northstar.example	+1 818-555-0149	Needs a client/contact CRM, pitch history, follow-up queue, and fast conversational

						retrieval.

15. Demo Workspace Dataset

The primary demo workspace should combine the above personas into one rich scenario named Northstar Demo Catalog. This gives a single login enough connected information to exercise nearly every core feature.

Song / recording	Artist	Rights state	Metadata state	Pitch state
Midnight Drive	Jordan Taylor	Writers 100%; Publishing 100%; Master 100%	Complete; one-stop ready; listen link	Currently pitching to Rebecca Carter; awaiting reply from a supervisor.
Ocean Blue	Jordan Taylor feat. Alex Ramirez	Writers 100%; Publishing 85%; Master 100%	Missing ISRC; listen link present	Not yet pitched; ideal validation demo.
Neon Static	Alex Ramirez / Jordan Taylor	Writers 100%; Publishing 100%; Master 75%	Complete composition metadata; incomplete master ownership	Currently pitching; should surface master warning.
Home Again	Jordan Taylor	Writers 100%; Publishing 100%; Master 100%	No licensing contact assigned	Has not been heard by several contacts.
Paper Planes at 2AM	Jordan Taylor	Writers 100%; Publishing 100%; Master 100%	Complete, includes lyrics and alternate title	Passed by one label, awaiting reply from sync agent.

Contact	Type	Company	Relevant pitch history
Rebecca Carter	Licensing Agent	Carter Sync Agency	Midnight Drive - Currently Pitching; Paper Planes at 2AM - Awaiting Reply
Maya Chen	Music Supervisor	Northlight Pictures	Midnight Drive - Awaiting Reply; Home Again - Hasn't Heard
Malik Johnson	Record Label	NextWave Records	Paper Planes at 2AM - Passed On It; Neon Static - Currently Pitching
Theo Grant	Music Supervisor	Blue Lantern Studios	Ocean Blue - Hasn't Heard; Home Again - Hasn't Heard

16. Demo Conversations That Must Work

ID	Prompt	Expected trace
DEMO-001	“Show me Midnight Drive.”	Mastra calls getComposition; CopilotKit renders CompositionDetail with real seeded data.
DEMO-002	“Who wrote this?”	Shared context resolves “this” to Midnight Drive and renders writer split details.

DEMO-003	“Which songs are not ready for sync?”	Agent queries deterministic readiness rules; renders problems grouped by song.
DEMO-004	“Why isn’t Ocean Blue ready?”	OwnershipValidator shows publishing total 85% and missing ISRC.
DEMO-005	“Show me everyone who hasn’t heard Home Again.”	Pitch query renders matching contacts, not a prose-only list.
DEMO-006	“Draft a follow-up to Rebecca for Paper Planes at 2AM.”	Draft uses pitch/contact/song context and opens EmailDraftEditor.
DEMO-007	“Send it.”	Mastra must suspend/interrupt for explicit approval; only then may the provider adapter run.
DEMO-008	“Change Alex’s writer split on Ocean Blue to 40%.”	Rights mutation requires confirmation, transaction, validation refresh, and audit event.
DEMO-009	“Generate a sync report without lyrics.”	Report engine uses canonical template/section rules and renders ReportPreview.
DEMO-010	“What should I follow up on today?”	Agent uses pitch dates/statuses/follow-up data and renders prioritized PitchCards.

17. Deterministic AIMock Fixture Plan for Demo

Fixture	Input pattern	Expected behavior
fx-show-song	show me midnight drive	Tool call getComposition({query:"Midnight Drive"}) then concise completion.
fx-pronoun-context	who wrote this	Uses shared state currentCompositionId; calls getCompositionRights.
fx-readiness	not ready for sync	Calls searchSyncReadiness({ready:false}); returns structured list.
fx-draft-followup	draft a follow-up	Calls getPitchContext then draftPitchEmail; no send side effect.
fx-send-approval	send it	Calls sendPitchEmail path that emits interrupt before provider execution.
fx-rights-mutation	change Alex split	Calls proposed mutation -> interrupt -> updateWriterSplit -> validateOwnership.
fx-report	report without lyrics	Calls generateMetadataReport with includeLyrics=false.

Frontend AGUIMock fixtures should separately emit tool-call and result events for each renderer. This verifies the React components independently from Mastra and from the LLM fixture behavior.

18. Test Pyramid and Required Gates

Layer	What it verifies	Representative tools
Unit	Pure validation, normalization,	Vitest; no DB/network.

	formatting, readiness calculations	
Service integration	SQL queries, transactions, authorization, workspace DB routing	Ephemeral/local libSQL or Turso test DB.
Mastra deterministic	Agent chooses correct tools/sequence from controlled model responses	AIMock + Vitest.
AG-UI component integration	Structured events render correct GIZMO components	AGUIMock + React test runner.
Full browser E2E	Better Auth session -> chat -> Mastra -> service -> Turso -> UI -> mutation	Playwright or equivalent.
Real-model evals	Prompt robustness and model quality beyond mocks	Evaluation suite, not required for every local test.

19. End-to-End Vertical Slice Gate

Before broad feature implementation, the architecture must prove one complete read and one sensitive write. If these fail, stop and correct architecture rather than building more screens.

Slice	Required path	Pass criteria
READ-VS-001	Better Auth login -> active workspace -> “Show me Midnight Drive” -> Mastra -> getComposition -> workspace Turso -> AG-UI -> CompositionDetail	Correct tenant data, structured UI, no prose-only substitute, deterministic AIMock replay passes.
WRITE-VS-001	“Change Alex’s split…” -> tool proposal -> authorization -> interrupt approval -> transaction -> revalidation -> AG-UI refresh -> audit record	No mutation before approval; correct DB updated; totals recomputed; UI reflects persisted state; cross-workspace denial test passes.

20. Architecture Decision Register

ADR	Decision	Status	Basis
ADR-001	Turso/libSQL replaces Supabase as canonical application database	Accepted	USER-DECISION
ADR-002	Database-per-workspace tenancy	Accepted pending implementation spike	Turso current architecture + project decision
ADR-003	Better Auth replaces previous/unspecified auth implementation	Accepted	USER-DECISION
ADR-004	Better Auth Organization is preferred workspace-membership primitive	Accepted pending repository integration	DOC-VERIFIED + architecture decision
ADR-005	Mastra is the GIZMO agent runtime	Accepted	USER-DECISION
ADR-006	CopilotKit/AG-UI owns conversational/generative application interface	Accepted	USER-DECISION + DOC-VERIFIED

ADR-007	AIMock/AGUIMock is required deterministic agent/UI test infrastructure	Accepted	USER-DECISION + DOC-VERIFIED
ADR-008	Mastra storage is logically separate from canonical GIZMO workspace data	Accepted	Architecture safety decision
ADR-009	Composition and recording are distinct entities	Accepted	DOMAIN-CORRECTION
ADR-010	LLM does not receive arbitrary SQL tool	Accepted	Security/safety decision
ADR-011	Object storage provider remains abstracted	Open vendor choice	UNRESOLVED
ADR-012	Email provider remains abstracted	Open vendor choice	UNRESOLVED

21. Items Still Not Verified

Item	Why still open	How to close it
Existing codebase compatibility with Better Auth	We have architecture intent, not repository proof.	Implement auth spike on current branch; generate/migrate Better Auth schema; session + org E2E test.
Exact Better Auth + libSQL adapter choice	Docs indicate SQLite/libSQL compatibility but project-specific adapter/runtime choice is not proven.	Choose Kysely/Drizzle/libSQL path and run migration + session integration test.
Turso workspace provisioning API implementation	Architecture is documented, not executed in this project.	Provision two test workspaces programmatically and run isolation test.
Schema migration fleet runner	Needed for database-per-workspace but not built.	Implement version registry, batch migration, retry/resume, dry-run, reporting.
Complete music-rights semantic validation	Legacy data model is known; authoritative domain review can still refine edge cases.	Validate IPI/CAE, PRO affiliation, publisher vs writer share, ISRC/ISWC/version rules with authoritative sources.
Email delivery/open/reply tracking provider	No provider frozen.	Select provider after contract interface is implemented.
Object storage provider	Turso should not store large binaries.	Select provider after AssetStorage interface and tests exist.
Full CopilotKit component behavior	Contracts specified but components are not yet implemented.	Vertical slice + AGUIMock renderer tests.
Real-model quality	AIMock proves deterministic behavior, not semantic quality.	Add evaluation set against chosen production models.

22. Build Order for the Next Agent

Phase	Deliverable	Exit gate

0	Repository audit; freeze versions; add requirement IDs to implementation plan	No assumptions about existing code remain.
1	Better Auth integration + organization/workspace control plane	AUTH E2E passes.
2	Turso workspace provisioning + migration runner + canonical schema	TEN isolation/provisioning tests pass.
3	Seed Northstar Demo Catalog and demo personas	All seed validation queries pass.
4	Catalog + rights services and conventional UI	CAT/RGT integration tests pass.
5	Mastra agent + safe read tools	READ-VS-001 passes with AIMock.
6	CopilotKit structured renderers + AGUIMock suite	No structured result regresses to prose-only UI.
7	Sensitive write + interrupt pattern	WRITE-VS-001 passes.
8	CRM, pitching, outreach drafts, report engine	Core CRM/OUT/RPT E2Es pass.
9	Email/storage adapters, Channels if desired, production hardening	External integration tests and threat review pass.

23. Coding-Agent Handoff Rules

- Do not replace Better Auth, Turso/libSQL, Mastra, CopilotKit/AG-UI, or AIMock without an explicit new user decision.
- Do not recreate Airtable one-for-one. Preserve verified business rules while implementing the canonical normalized domain model.
- Do not create an executeSQL tool for the LLM. All database access must pass through typed, authorized GIZMO services.
- Do not store canonical business facts only in agent memory. Turso workspace data is the source of truth.
- Do not treat AIMock as a substitute for real-model evaluation. Use it for deterministic regression and CI.
- Do not mark a requirement complete until its mapped test is implemented and passing.
- Every pull request should list the requirement IDs it implements or changes and the tests that prove them.
- When implementation reveals a contradiction, update the ADR/traceability contract rather than silently diverging.

24. Source Register (current as of September 2026)

Ref	Source	URL
S1	Better Auth - Introduction / current version and capabilities	https://better-auth.com/docs/introduction
S2	Better Auth - Organization plugin	https://better-auth.com/docs/plugins/organization
S3	Better Auth - SQLite adapter	https://better-auth.com/docs/adapters/sqlite
S4	Better Auth - Other relational databases / libSQL-sqld dialect listing	https://better-auth.com/docs/adapters/other-relational-databases
S5	CopilotKit - Mastra integration overview	https://docs.copilotkit.ai/mastra
S6	CopilotKit - Mastra Tool Rendering	https://docs.copilotkit.ai/mastra/generative-ui/

		tool-rendering
S7	CopilotKit - Mastra Shared State	https://docs.copilotkit.ai/mastra/shared-state
S8	CopilotKit - AG-UI Overview	https://docs.copilotkit.ai/ag-ui/introduction
S9	CopilotKit - Mastra interrupts / human-in-the-loop	https://docs.copilotkit.ai/mastra/human-in-the-loop/useInterrupt
S10	AIMock - Mastra integration	https://aimock.copilotkit.dev/integrate-mastra/
S11	AIMock - Documentation	https://aimock.copilotkit.dev/docs/
S12	Turso - Multi-tenancy at Scale	https://turso.tech/blog/multi-tenancy-at-scale
S13	Turso - Multi-DB Schemas (Deprecated)	https://docs.turso.tech/features/multi-db-schemas
S14	Mastra - Managed Databases / LibSQLStore	https://mastra.ai/blog/introducing-managed-databases

25. Final Technical Traceability Gate

Definition of done for the architecture
GIZMO is ready for broad production implementation when Better Auth workspace resolution, Turso database isolation/provisioning, canonical catalog/rights queries, Mastra tool invocation, CopilotKit rendering, AIMock/AGUIMock regression tests, and one approved write mutation all pass as a single traced vertical slice. Until then, those areas are specifications rather than proven architecture.

This document intentionally preserves uncertainty. A coding agent should be able to distinguish what is proven, what is specified, and what still requires implementation evidence. That distinction is the purpose of technical traceability.